

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	GANESH M	Reg. No.:	192524034
Section / Batch:	B	Date:	19-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies.

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I GANESH M / 192524034_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

CO4 –AT-1 CODING CHALLENGE TASK

Student Name: Ganesh M

Register Number: 192524034

Q13. Coin Change Problem (Minimum Coins)

A cashier must provide change for a given amount using a set of available coin denominations. The aim is to determine the minimum number of coins required to make the exact amount, where each denomination can be used multiple times. If the amount can be formed in several ways, the solution with the fewest coins must be selected. Use Dynamic Programming to compute the optimal answer.

Sample Input

n = 3

coins = 1 2 5

amount = 11

Expected Output

Minimum Coins = 3

1. AIM

The aim is to determine the minimum number of coins required to form a given amount exactly using Dynamic Programming. Each coin denomination may be used multiple times, and the solution with the fewest coins is selected.

2. PROBLEM STATEMENT

A cashier must provide change for a given amount using a set of available coin denominations. The objective is to find the minimum number of coins needed to make the exact amount. Each denomination can be used any number of times. If multiple combinations can form the amount, the combination using the fewest coins must be selected.

3. PROBLEM UNDERSTANDING

- Input: number of coin denominations, coin values, and the target amount.
- Output: minimum number of coins required to form the target amount exactly.
- Each denomination can be used multiple times.
- The exact target amount must be formed.
- Among all valid combinations, the solution with the fewest coins is selected.
- Important edge cases include amount = 0 and an amount that cannot be formed.

4. OBJECTIVES

- Apply Dynamic Programming to solve an optimization problem.
- Use optimal substructure to build the minimum-coin solution.
- Select the minimum number of coins for the exact target amount.
- Avoid repeated calculations by storing previously computed results.
- Handle normal, boundary, impossible, and larger test cases.

5. ALGORITHM

1. Read n , the number of coin denominations, the coin values, and the target amount.
2. Create a DP array $dp[0 \dots \text{amount}]$.
3. Set $dp[0] = 0$ and initialize all other values to a large value.
4. For every amount i from 1 to the target amount, check every coin denomination.
5. If $\text{coin} \leq i$, update $dp[i] = \min(dp[i], dp[i - \text{coin}] + 1)$.
6. After filling the DP array, check $dp[\text{amount}]$.
7. If it is still the large value, print -1; otherwise print $dp[\text{amount}]$.

6. PSEUDOCODE

```
START
Read n
Read coins
Read amount

Create dp[0...amount]
Set dp[0] = 0
Set all other dp values to INF

For i = 1 to amount:
    For each coin c:
        If c <= i:
            dp[i] = min(dp[i], dp[i-c] + 1)

If dp[amount] == INF:
    Print -1
Else:
    Print "Minimum Coins =", dp[amount]
END
```

7. PYTHON PROGRAM

```
import tkinter as tk

r=tk.Tk()

r.title("Coin Change DP Game")

r.geometry("700x500")

r.config(bg="#17202A")

s=-1

t=["CHALLENGE","CREATE DP","CALCULATE","OPTIMAL SOLUTION","FINAL RESULT"]

x=["Coins: 1,2,5 | Amount: 11","dp[0]=0 | Others=INF",

   "dp[i]=min(dp[i],dp[i-c]+1)","5+5+1 = 11 | 3 coins",

   "MINIMUM COINS = 3"]

def show():

    c.delete("all")

    if s<0:

        c.create_text(350,200,text="READY?",font=("Arial",35,"bold"),fill="yellow")

    return

    c.create_text(350,60,text="STEP "+str(s+1)+" - "+t[s],

                  font=("Arial",20,"bold"),fill="white")

    c.create_rectangle(100,130,600,230,fill="white")

    c.create_text(350,180,text=x[s],font=("Arial",18,"bold"),fill="#17202A")

    c.create_text(350,300,text="NEXT → learn the algorithm",

                  font=("Arial",16,"bold"),fill="yellow")

def start():

    global s

    s=0

    show()

def next():

    global s if s<4:s+=1 show()
```

```

def prev():
    global s
    if s>0:s-=1
    show()
def finish():
    global s
    s=4
    show()
def restart():
    global s
    s=-1
    show()
tk.Label(r,text="COIN CHANGE - DP GAME",
        font=("Arial",24,"bold"),bg="#17202A",fg="white").pack(pady=15)
c=tk.Canvas(r,width=650,height=350,bg="#212F3D")
c.pack()
f=tk.Frame(r,bg="#17202A")
f.pack(pady=10)
for text,fun in [("START",start),("PREVIOUS",prev),("NEXT",next),
                ("FINISH",finish),("RESTART",restart)]:
    tk.Button(f,text=text,command=fun,width=10).pack(side="left",padx=3)
show()
r.mainloop()

```

8. SAMPLE INPUT AND OUTPUT

Sample Input:

```

3
1 2 5
11

```

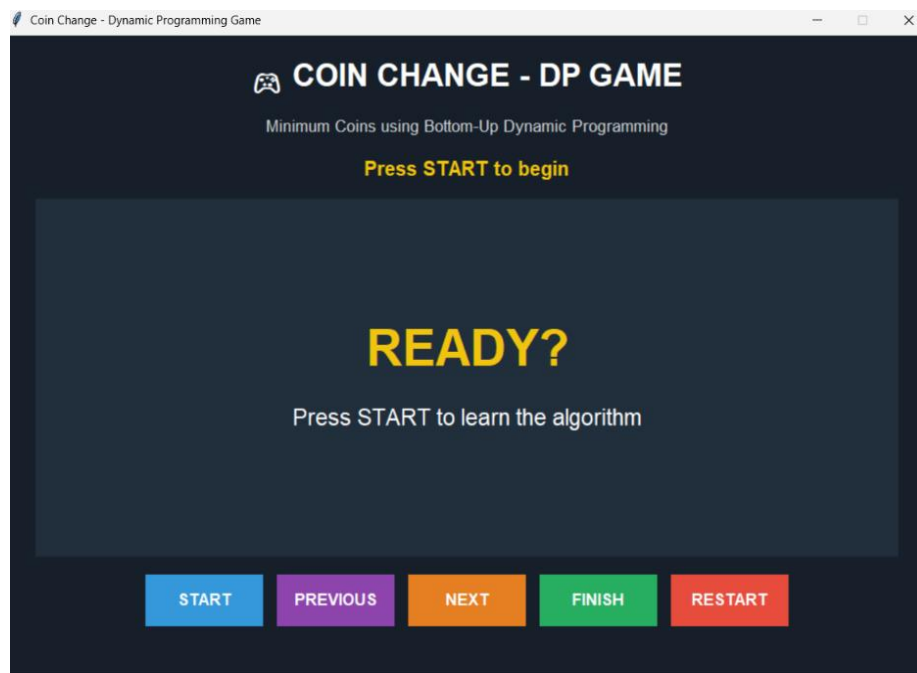
Expected Output:

Minimum Coins = 3

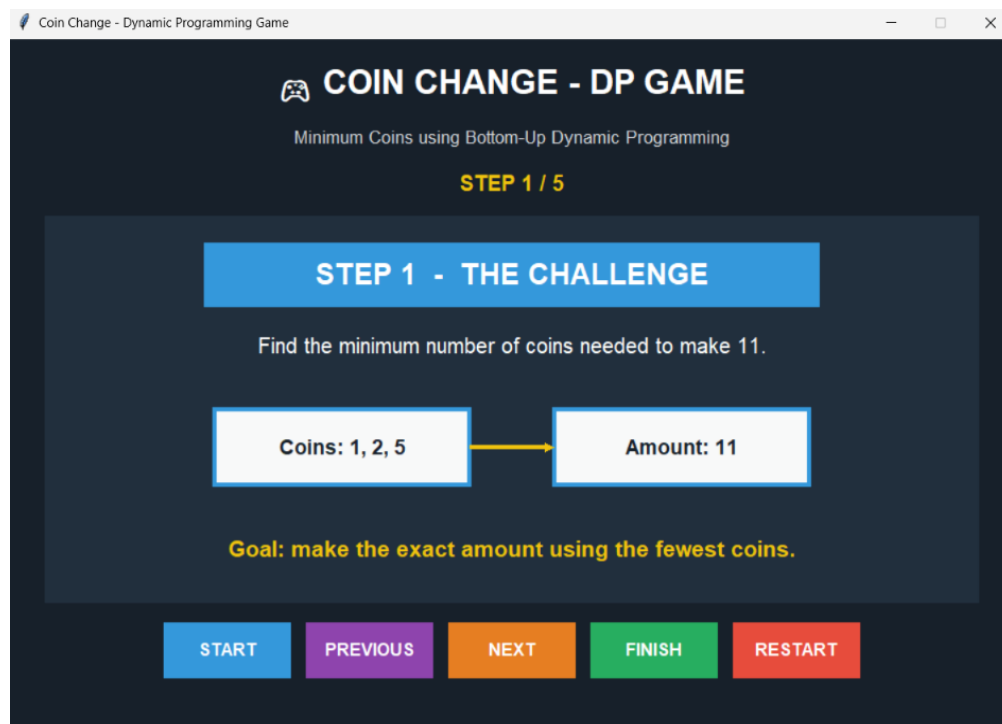
9. PROGRAM IMPLEMENTATION SCREENSHOTS

The following screenshots show the Python Tkinter interactive visualization created for the Coin Change Dynamic Programming algorithm.

Screenshot 1 – Initial Game Screen



Screenshot 2 – Step 1: The Challenge



Screenshot 3 – Step 2: Create DP

COIN CHANGE - DP GAME
Minimum Coins using Bottom-Up Dynamic Programming

STEP 2 / 5

STEP 2 - CREATE DP

$dp[i]$ stores the minimum coins needed to make amount i .

$dp[0] = 0$ → Other values = INF

The DP table remembers answers for smaller amounts.

START PREVIOUS NEXT FINISH RESTART

Screenshot 4 – Step 3: Calculate

COIN CHANGE - DP GAME
Minimum Coins using Bottom-Up Dynamic Programming

STEP 3 / 5

STEP 3 - CALCULATE

Try every coin and keep the smallest possible value.

$dp[i] = \min(dp[i], dp[i-c] + 1)$ → Check coins: 1, 2, 5

Each new amount uses previously calculated DP values.

START PREVIOUS NEXT FINISH RESTART

Screenshot 5 – Step 4: Optimal Solution

The screenshot shows a web application titled "COIN CHANGE - DP GAME" with the subtitle "Minimum Coins using Bottom-Up Dynamic Programming". The progress indicator shows "STEP 4 / 5". The main content area has a green header "STEP 4 - OPTIMAL SOLUTION". Below it, the text says "For 11, the best combination is 5 + 5 + 1." A diagram shows a box "5 coins -> too many" with an arrow pointing to a box "5 + 5 + 1 -> 3 coins". Below the diagram, the text reads "5 + 5 + 1 = 11 → only 3 coins." At the bottom, there are five buttons: "START" (blue), "PREVIOUS" (purple), "NEXT" (orange), "FINISH" (green), and "RESTART" (red).

Screenshot 6 – Step 5: Final Result

The screenshot shows the same web application, now at "STEP 5 / 5". The main content area has a teal header "STEP 5 - FINAL RESULT". Below it, the text says "Dynamic Programming gives the optimal answer." A diagram shows a box "AMOUNT = 11" with an arrow pointing to a box "MINIMUM COINS = 3". Below the diagram, the text reads "🏆 Optimal answer found: 3 coins!". At the bottom, there are five buttons: "START" (blue), "PREVIOUS" (purple), "NEXT" (orange), "FINISH" (green), and "RESTART" (red).

10. INTERACTIVE VISUALIZATION EXPLANATION

- The Tkinter application presents the algorithm as a five-step game-like visualization.
- START begins the visualization and displays Step 1.
- NEXT reveals the next algorithm step.
- PREVIOUS moves back to the previous step.
- FINISH jumps directly to the final result.
- RESTART returns the application to the initial READY screen.
- The final screen clearly displays Amount = 11 and Minimum Coins = 3.

11. DP TABLE / WORKING

For coins 1, 2, and 5, the DP values for amounts 0 through 11 are:

0	1	2	3	4	5	6	7	8	9	10	11
0	1	1	2	2	1	2	2	2	3	2	3

For amount 11, the minimum is 3 coins: $5 + 5 + 1 = 11$.

12. TEST CASES

Case	Input	Expected Output	Explanation
1	3 / 1 2 5 / 11	Minimum Coins = 3	5 + 5 + 1 uses 3 coins.
2	3 / 1 2 5 / 0	Minimum Coins = 0	Zero coins are needed.
3	1 / 7 / 14	Minimum Coins = 2	7 + 7 forms 14.
4	2 / 2 4 / 3	-1	No combination can form 3.
5	3 / 1 5 10 / 27	Minimum Coins = 5	10 + 10 + 5 + 1 + 1 uses 5 coins.

13. TIME COMPLEXITY

Time Complexity = $O(n \times \text{amount})$, where n is the number of coin denominations. Each amount is processed against every denomination.

14. SPACE COMPLEXITY

Space Complexity = $O(\text{amount})$, because a one-dimensional DP array of size amount + 1 is used.

16. EDGE CASE HANDLING

- Amount = 0: the result is 0.
- No possible combination: the program prints -1.
- Coin greater than the target amount: it is ignored for that amount.
- Repeated use of denominations: supported by the DP recurrence.
- Duplicate or unnecessary denominations: they do not change the minimum result.

17. RESULT

The Dynamic Programming algorithm successfully finds the minimum number of coins required to form the given amount. For coins 1, 2, 5 and amount 11, the result is Minimum Coins = 3.

18. CONCLUSION

The Coin Change problem is an optimization problem that can be solved effectively using Dynamic Programming. The algorithm stores minimum results for smaller amounts and uses them to construct the optimal solution. The Tkinter visualization presents these steps interactively and makes the algorithm easier to understand. For the given sample, the minimum number of coins required is 3.

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

